

🔑 Objectifs de la feuille

- Les vues génériques
- La vue générique de base
- La vue générique d'affichage
- La vue générique de détail
- Navigation & modèle d'URL
- Premier formulaire DJANGO

Les vues génériques

Les vues `ListProduits()`, `ListCategories()`, `ListStatuts()` et `ListRayons()` sont très courtes et que dire des vues `about()`, `home()` et `contact()`. Très rapidement vous vous rendrez compte que vos vues se ressemblent toutes plus ou moins.

Ces vues représentent un cas classique du développement Web : récupérer les données depuis la base de données suivant un paramètre contenu dans l'URL, charger un gabarit et renvoyer le gabarit interprété. Ce cas est tellement classique que DJANGO propose un raccourci, appelé le système de vues génériques.

Les vues génériques ajoutent une couche d'abstraction des procédés courants au point où vous n'avez même plus besoin d'écrire du code Python pour écrire une application. Finalement on voudra lister des modèles, puis pouvoir en créer, modifier et enfin supprimer. Ainsi, lorsque nous construisons l'interface utilisateur de notre site, on peut dire que nous construisons une interface CRUD.

Le concept de DJANGO c'est de faciliter au maximum la conception d'un projet. Inutile de répéter du code, on se veut DRY et factoriser le code est aussi intéressant sur la performance du développeur que sur la maintenance.

Nous allons convertir notre application pour qu'elle utilise le système de vues génériques. Nous pourrons ainsi supprimer une partie de notre code. Nous avons quelques pas à faire pour effectuer cette conversion. Nous allons :

1. Convertir les urls
2. Supprimer quelques anciennes vues désormais inutiles.
3. Introduire de nouvelles vues basées sur les vues génériques de DJANGO.

Pourquoi ces changements de code ?

En général, lorsque vous écrivez une application DJANGO, vous devez estimer si les vues génériques correspondent bien à vos besoins et, le cas échéant, vous les utiliserez dès le début, plutôt que de réarranger votre code à mi-chemin. Mais ce cours s'est concentré intentionnellement sur l'écriture des vues «à la dure» jusqu'à ce TD, pour mettre l'accent sur les concepts de base. Tout comme vous devez posséder des bases de maths avant de commencer à utiliser une calculatrice.

La vue générique de base

Commençons par la vue générique de base: `TemplateView`. Cette vue se contente de renvoyer un fichier type static, pas de modèles, pas de requêtes en base de données. Nous allons appliquer cette vue générique pour notre «home». Pour cela, il faut agrémenter le fichier `monApp/urls.py` d'une nouvelle url comme ci-dessous (vous mettrez l'ancienne url en commentaire) :

```
from django.urls import path
from . import views
from django.views.generic import *

urlpatterns = [
    #path("home", views.home, name="home"),
    ...
    path("home", TemplateView.as_view(template_name="monApp/page_home.html")),
]
```

Dans cet exemple, le template `home.html` sera utilisé comme nous l'avons indiqué. Nul besoin d'instancier quoi que ce soit, la classe `TemplateView` ne nécessite que l'appel de la méthode `as_view`. Comme vous le voyez, il ne se passe plus rien dans le fichier `view.py`. La vue n'est plus basée sur une fonction mais sur une classe.

- Les attributs de la classe `TemplateView`
`content_type = None`
`http_method_names = ['get', 'post', 'put', 'patch', 'delete', 'head', 'options', 'trace']`
`response_class = <class 'django.template.response.TemplateResponse'>`
`template_name = None`
- Les méthodes de la classe `TemplateView`
`_allowed_methods(self)`
`as_view(cls, **kwargs)`
`dispatch(self, request, *args, **kwargs)`
`get(self, request, *args, **kwargs)`
`get_context_data(self, **kwargs)`
`get_template_names(self)`
`http_method_not_allowed(self, request, *args, **kwargs)`
`options(self, request, *args, **kwargs)`
`render_to_response(self, context, **response_kwargs)`

En revanche, notre message « Hello Django ! » ne s'affiche plus (et nos tests sont KO) car nous ne donnons rien à la vue dans le dictionnaire de contexte. Nous allons donc baser notre vue `home` sur une classe qui hérite de `TemplateView` afin d'utiliser la méthode `get_context_data()` pour lui donner du contexte.

La vue `home` dans le fichier `views.py` devient :

```
class HomeView(TemplateView):
    template_name = "monApp/page_home.html"

    def get_context_data(self, **kwargs):
        context = super(HomeView, self).get_context_data(**kwargs)
        context['message'] = "Hello Django!"
        return context
```

L' url dans le fichier `monApp/urls.py` devient : `path("home/", views.HomeView.as_view())`,
Et voilà, nous voyons notre message !

C'est à vous de jouer !

- nous nous servons du même template pour les vues `about` et `contact`, alors faites de même pour ces deux vues.
- en gardant le même template `page_home.html`, modifiez votre vue `home` et la vue dont l'url est `home/<param>` pour que la vue `home/Cricri` affiche « Hello Cricri ». Vous aurez besoin de ce bout de code `self.kwargs.get('param')`. A vous de mettre cette instruction au bon endroit.

A la fin de cet exercice, vous devez avoir :

- 3 vues basées sur des classes
- un seul template `page_home.html` possédant dans son contexte :
 - une variable pour le paramètre `param` de l'url `home/<param>`
 - une variable pour le message pour les trois urls `home/`, `about/` et `contact/`.
- supprimez vos anciens vues basées sur les fonctions : `def home`, `def contact`, `def about`
- maintenez votre taux de couverture de test

La vue générique d'affichage

L'une des vues génériques d'affichage les plus utilisées est `ListView`. Cette vue a vocation à travailler sur une liste d'objets. C'est le même principe que précédemment. La fonction `ListProduits()` dans le fichier `views.py` va être remplacée par la classe `ProduitListView` qui va hériter de `ListView`. Il lui faut un modèle, un template et du contexte :

```
class ProduitListView(ListView):  
    model = Produit  
    template_name = "monApp/list_produits.html"  
    context_object_name = "produits"
```

Et il ne faut oublier le nouveau path de l'url : `path("produits/", views.ProduitListView.as_view())`,

Vous pouvez bien évidemment choisir l'emplacement et le nom de votre template, ainsi que le nom de l'objet passé dans le contexte.

Il est possible de modifier la requête qui liste les produits avec le param `quesryset`:

```
class ProduitListView(ListView):  
    model = Produit  
    template_name = "monApp/list_produits.html"  
    context_object_name = "produits"  
    queryset = Produit.objects.filter(refProd=2)  # filtre
```

ou en passant par la méthode `get_queryset()` :

```
class ProduitListView(ListView):  
    model = Produit  
    template_name = "monApp/list_produits.html"  
    context_object_name = "prdts"  
  
    def get_queryset(self):  
        return Produit.objects.order_by("prixUnitaireProd") # tri sur le prix
```

Si vous voulez passer d'autres données, vous pouvez passer par la méthode `get_context_data()` dans la classe `ProduitListView`:

```
def get_context_data(self, **kwargs):  
    context = super(ProduitListView, self).get_context_data(**kwargs)  
    context["titremenu"] = "Liste de mes produits !"  
    return context
```

Il vous suffit de récupérer votre variable `{{titremenu}}` dans votre template dans le bloc `menu` !

La vue générique de détail

`DetailView` est identique à `ListView` sauf qu'il s'agit que d'un seul item ; le path de l'url est donc :

```
path("produit/<pk>/", views.ProduitDetailView.as_view()),
```

La classe `ProduitDetailView` hérite de `DetailView` :

```
class ProduitDetailView(DetailView):
    model = Produit
    template_name = "monApp/detail_produit.html"
    context_object_name = "prdt"

    def get_context_data(self, **kwargs):
        context = super(ProduitDetailView, self).get_context_data(**kwargs)
        context['titremenu'] = "Détail du produit"
        return context
```

Nous définissons alors un nouveau template `detail_produit.html` :

```
{% extends 'monApp/base.html' %}

{% block menu %}
<h1>{{titremenu}}</h1>
{% endblock %}

{% block contenu %}
<h2>{{ prdt.intituleProd }}</h2>

<ul>
    <li>Référence produit : {{prdt.refProd }} </li>
    <li>Prix unitaire HT : {{ prdt.prixUnitaireProd }} </li>
    <li>Date de fabrication : {{ prdt.dateFabProd }} </li>
    <li>Catégorie : {{ prdt.categorie }} </li>
    <li>Statut : {{ prdt.statut}}</li>
</ul>
{% endblock %}
```

Et le tour est joué !

Navigation et modèle d'URL

C'est bien que nous puissions accéder à la page du produit ipod en visitant `http://127.0.0.1:8000/monApp/produit/1/`, mais attendons-nous vraiment de nos utilisateurs qu'ils sachent que «1» est l'identifiant du produit ipod et qu'ils le tapent dans la barre d'adresse ? Bien sûr que non. De nombreux utilisateurs ne regardent même pas la barre d'adresse lorsqu'ils naviguent sur un site. Le fait que certains navigateurs modernes comme Safari cachent désormais le chemin d'accès à l'URL dans la barre d'adresse en est la preuve !

La plupart des utilisateurs s'attendent implicitement à être guidés sur le site par des éléments de navigation intuitifs, comme les hyperliens. Mettons à jour le modèle de la vue en liste, afin que chaque nom de produit devienne un lien cliquable qui nous amène à la vue détaillée de ce groupe.

```
<a href="/produit/{{prdt.id}}">{{ prdt.intituleProd }}</a>
```

Mais vous ne devriez pas faire ça ! C'est un anti-pattern.

Cela va générer un lien fonctionnel : n'hésitez pas à tester ! Mais c'est un anti-pattern. Pourquoi ? Parce que c'est du code répété : nous construisons déjà ce chemin d'une manière similaire dans `urls.py` :

```
path("produit/<pk>/", views.ProduitDetailView.as_view()),
```

Si nous voulions changer le chemin d'accès à la vue détaillée, par exemple, en «`tous_les_produits/1/` », nous devrions le modifier dans `urls.py` *et* dans chaque gabarit où nous répétons le lien. Comment éviter la répétition du code et rendre nos liens plus faciles à maintenir ? Commençons par donner au modèle d'URL un argument `name`:

```
path("produit/<pk>/", views.ProduitDetailView.as_view(), name="detailProduit"),
```

J'ai utilisé `detailProduit` pour le nom de la vue ici, mais le `name` peut être ce que vous voulez.

Le modèle d'URL est maintenant notre «source unique de vérité» pour ce lien. Pour générer ce lien dans notre template `list_produits.html`, nous allons utiliser une balise de gabarits sur le champ `refProd` :

```
<td><a href="{% url 'detailProduit' prdt.refProd %}">{{ prdt.refProd }}</a></td>
```

Nous passons deux arguments à la balise `url` :

- `detailProduit`
- `prdt.refProd`.

C'est comme passer des arguments à une fonction Python, mais dans les balises de gabarits, et nous n'avons pas besoin de saisir les parenthèses. Maintenant, si nous rafraîchissons notre liste de produits, nous pouvons voir que chaque nom de produit est maintenant un lien, et nous pouvons cliquer sur la vue détaillée du produit.

Faisons maintenant la même chose dans l'autre sens : la vue détaillée du groupe doit comporter un lien vers la liste des groupes. Dans `monApp/urls.py` nous avons le name `listProduits`:

```
path("produits/",views.ProduitListView.as_view(),name="listProduits"),
```

Et le template `detail-produit.html` reçoit une ligne supplémentaire :

```
<a href="{% url 'listProduits' %}">Retour à la liste des produits</a>
```

Puisque nous travaillons avec les liens, profitons-en pour ajouter une barre de navigation à notre site, ainsi qu'un lien vers la liste des produits. Nous l'ajouterons dans le gabarit `base.html`, en tant que premier élément de la balise `<body>`, car la barre de navigation doit apparaître en haut de chaque page :

```
{% block nav %}
<nav class="navbar navbar-expand-md navbar-dark bg-dark mb-4">
  <div class="collapse navbar-collapse" id="navbarNav">
    <ul class="navbar-nav ml-auto">
      <li class="nav-item">
        <a class="nav-link" href="{% url 'listProduits' %}">Produits</a>
      </li>
    </ul>
  </div>
</nav>
{% block menu %}{% endblock %}
{% endblock %}
```

C'est à vous de jouer !

Maintenant, c'est à votre tour de créer des vues de liste et de détail basées sur des classes pour le modèle `Categorie`, `Statut` et `Rayon` et d'ajouter des liens entre les deux. Ajoutez également un lien vers la vue de liste dans notre barre de navigation pour chacun d'eux. Et comme toujours, maintenez votre taux de couverture de tests !

Premier formulaire DJANGO

Nous allons apprendre par la suite comment créer un formulaire dans les règles de l'art DJANGO et comment les utiliser dans DJANGO. Les formulaires DJANGO vont nous présenter quelques nouveaux concepts. Nous allons donc apprendre à créer un formulaire de « Contact » qui permet aux utilisateurs d'envoyer un message aux administrateurs de l'application.

Nous commençons par définir le formulaire comme une classe. Créez le fichier « `monApp/forms.py` » et ajoutez ce code :

```
from django import forms

class ContactUsForm(forms.Form):
    name = forms.CharField(required=False)
    email = forms.EmailField()
    message = forms.CharField(max_length=1000)
```

Nous avons défini trois champs de formulaire dans notre `ContactUsForm`. Les champs de formulaire sont similaires aux champs de modèle : nous avons différents types de champs pour différents types de données. Nous pouvons également préciser quand les champs doivent être facultatifs avec `required=False`. Ici, nous permettons à l'utilisateur de rester anonyme en rendant le champ `name` facultatif. Et nous pouvons définir `max_length`, tout comme nous le faisons dans un modèle.

Ensuite, utilisons notre `ContactUsForm` dans la vue `ContactView` basée sur une fonction. Autrement dit, mettez le code de la vue `ContactView` basée sur la classe `TemplateView` en commentaires et écrivez ce qui suit à la place :

```
def ContactView(request):
    form = ContactUsForm()
    message = "Contact Us"
    return render(request, "monApp/page_home.html", {'message':message, 'form':form})
```

Remettez à jour votre fichier `urls.py` avec la ligne suivante :

```
path("contact/", views.ContactView, name="contact"),
```

Et maintenant, dans notre gabarit `page_home.html`, nous allons ajouté une balise `<form>` avec un `<input>` de `type="submit"` à l'intérieur. C'est la norme pour tout formulaire HTML que vous pouvez créer. Mais maintenant les choses deviennent intéressantes. Au lieu de taper manuellement les balises `<input>` pour chacun des champs de notre formulaire, nous tapons simplement `{{ form }}`. Nous ajoutons également `{% csrf_token %}`, une protection facile à mettre en place contre les attaques de type CROSS SITE. Ces types d'attaques consiste à injecter du code malicieux à travers le bouton d'un formulaire.

Le bloc `contenu` du fichier `page_home.html` devient :

```
{% block contenu %}
    {% if param %}
        <h1>Hello {{ param }}!</h1>
    {% else %}
        <h1>{{message}}</h1>
    {%endif%}
    {% if form %}
        <form action="" method="post" novalidate>
            {% csrf_token %}
            {{ form }}
            <button type="submit" class="btn btn-success mt-2">Envoyer</button>
        </form>
    {%endif%}
{% endblock %}
```

Jetons un coup d'œil à cela dans le navigateur à l'adresse <http://127.0.0.1:8000/monApp/contact/> pour comprendre ce qui se passe ici, et examinons le HTML généré dans les outils de développement de notre navigateur. Nous pouvons voir que DJANGO a automatiquement généré un `<label>` et un `<input>` pour chacun de nos champs de formulaire : name, email et message. C'est génial : cela signifie que chaque fois que nous voulons ajouter un nouveau champ à ce formulaire, nous l'ajoutons simplement à la classe `ContactUsForm`, et DJANGO s'occupera du HTML pour nous. Une petite modification s'impose dans le gabarit : les champs seraient plus beaux s'ils étaient alignés :

```
{{ form.as_p }}
```

Cela encadre chaque paire balise de champ dans une balise `<p>`.

Regardez à nouveau la balise `<form>` que nous avons utilisée :

- `novalidate` désactive la validation de formulaire de votre navigateur. Il s'agit d'une fonctionnalité utile de la plupart des navigateurs, et nous la réactiverons plus tard, mais nous devons d'abord vérifier que notre formulaire fonctionne correctement sans elle.
- La valeur de `method` est `post` : ce qui signifie que les données seront envoyées comme une requête HTTP POST. C'est un peu différent des requêtes GET que nous avons utilisées jusqu'à présent, car en plus d'une «méthode» et d'un «chemin», elle comprend également un «corps» de requête qui contient les données du formulaire.
- L'attribut `action` désigne l'URL où nous allons envoyer les données du formulaire. Si vous donnez à cet attribut la valeur d'une chaîne vide, il renverra à l'URL de la page où nous nous trouvons déjà, c'est-à-dire `"http://127.0.0.1:8000/monApp/contact/"`. Cela signifie que nous allons gérer les données du formulaire dans notre vue `contact`.

Maintenant que nous savons que les données de notre formulaire arriveront au serveur sous forme de requête POST, voyons comment gérer les requêtes POST dans notre vue. Pour avoir un aperçu plus clair du fonctionnement des données POST dans une vue, nous allons utiliser un peu de journalisation dans le terminal, avec des instructions `print` :

```
def ContactView(request):  
    form = ContactUsForm()  
    titre1 = "Contact us !"  
    print('La méthode de requête est : ', request.method)  
    print('Les données POST sont : ', request.POST)  
    return render(request, "monApp/page_home.html", {'titre1': titre1, 'form': form})
```

Tout d'abord, appelons cette vue comme une requête GET. Le moyen le plus sûr de le faire est de cliquer dans la barre d'adresse du navigateur et d'appuyer sur **Entrée**. Maintenant, regardez dans le terminal :

La méthode de requête est : GET
Les données POST sont : <QueryDict: {}>

Nous pouvons voir que pendant une requête GET, `request.POST` est un QueryDict vide (qui est un type spécial de `dict` Python). Maintenant, demandons la même vue en tant que requête POST. Pour ce faire, nous saisissons des données dans les champs du formulaire, puis nous cliquons sur «Envoyer». Cette fois, nous pouvons voir que notre `QueryDict` contient les données de notre formulaire ! (Y compris le mystérieux jeton CSRF).

La méthode de requête est : POST
Mes données POST sont :
<QueryDict:{
'csrfmiddlewaretoken': 'gG1HEKfwmqlh9nTE8mbdg5X9Vtcjz bmdWCXOgzDw71qVGLWyaN2DSnq8Wjm
NWWml'],
'name': ['titof'], '
email': ['titof@gmail.com'],
'message': ['bonjour']

Notez qu'après que nous avons soumis les données et que la vue contact s'est rechargée, nous voyons à nouveau un formulaire vide, parce qu'actuellement notre vue crée simplement un nouveau formulaire vide à chaque fois qu'elle s'exécute (`form = ContactUsForm()`).

Ensuite, nous devons d'une manière ou d'une autre gérer les deux scénarios de demande dans notre vue :

- S'il s'agit d'une requête GET, nous devons afficher un formulaire vide à l'utilisateur.
- S'il s'agit d'une demande POST, nous devons examiner les données et voir si elles sont valides.

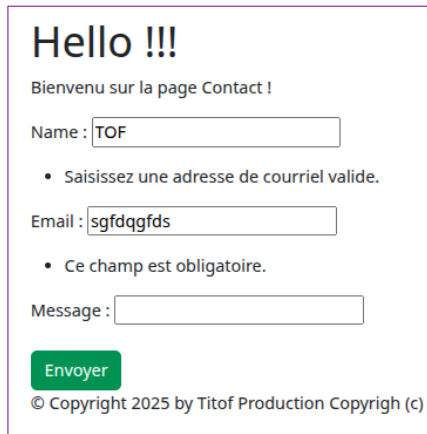
Nous avons donc besoin d'une instruction `if`:

```
def ContactView(request):
    titre1 = "Contact us !"

    if request.method=='POST':
        form = ContactUsForm(request.POST)
    else:
        form = ContactUsForm()

    return render(request, "monApp/page_home.html",{ 'titre1':titre1, 'form':form})
```

Dans les deux branches de l'instruction `if`, nous créons un formulaire qui est transmis au modèle, mais dans le cas d'une requête `POST`, nous remplissons également le formulaire avec les données `POST`. Maintenant, lorsque nous soumettons un formulaire rempli, les données sont toujours visibles dans le navigateur lorsque la page est rechargée. Mais ce n'est pas tout : si nous soumettons des données non valides, le formulaire affichera des messages d'erreur.



The screenshot shows a web form titled "Hello !!!" with the subtitle "Bienvenu sur la page Contact !". It contains three input fields: "Name" with the value "TOF", "Email" with the value "sgfdqgfs", and "Message" which is empty. There are two error messages: "Saisissez une adresse de courriel valide." (Enter a valid email address) next to the email field, and "Ce champ est obligatoire." (This field is required) next to the message field. A green "Envoyer" (Send) button is at the bottom. The footer reads "© Copyright 2025 by Titof Production Copyrigh (c)".

Ce que nous voyons ici est une validation côté serveur : notre formulaire DJANGO a validé les champs par rapport à nos règles, a généré des messages d'erreur en cas de problème, puis les a retournés dans le modèle comme faisant partie du formulaire. L'utilisateur peut alors modifier les valeurs et soumettre à nouveau le formulaire, et DJANGO vérifiera à nouveau le formulaire. Ce cycle peut se répéter autant de fois que nécessaire jusqu'à ce que tous les champs du formulaire soient valides. A ce stade, nous sommes enfin prêts à effectuer l'action que nous voulions faire en premier lieu : envoyer un e-mail !

Nous avons maintenant un peu plus de logique à mettre en œuvre dans notre vue :

- Si c'est une requête `POST`...
 - Si les données du formulaire sont valides, envoyer un e-mail ;
 - Si les données du formulaire ne sont pas valides, afficher à nouveau le formulaire avec des messages d'erreur (comme nous le faisons déjà).

Nous avons besoin d'une instruction `if` imbriquée :

```
from django.core.mail import send_mail

def ContactView(request):
    titre1 = "Contact us !"
    if request.method=='POST':
        form = ContactUsForm(request.POST)
        if form.is_valid():
            send_mail(
                subject=f'Message from {form.cleaned_data["name"]} or "anonyme" via monProj
                Contact form',
                message=form.cleaned_data['message'],
                from_email=form.cleaned_data['email'],
                recipient_list=['admin@monApp.com'],
            )
        else:
            form = ContactUsForm()
    return render(request, "monApp/page_home.html", {'titre1':titre1, 'form':form})
```

Nous importons la fonction `send_mail` de DJANGO au début. Ensuite, nous insérons l'instruction imbriquée, commençant par : `if form.is_valid()`. Si tous les champs de notre formulaire contiennent des données valides, alors `form.is_valid()` renvoie `True` et nous appelons `send_mail` pour envoyer notre e-mail.

`form.cleaned_data` est un `dict` contenant les données du formulaire après qu'elles ont subi le processus de validation. Lorsque nous sommes prêts à faire quelque chose avec les données de notre formulaire, nous pouvons accéder à chacun des champs via `form.cleaned_data['name_of_field']`, mais nous devons d'abord appeler `form.is_valid()`.

L'envoi d'un véritable e-mail implique la configuration d'un serveur SMTP, que nous n'avons malheureusement pas le temps de couvrir ici ! Mais nous pouvons utiliser le serveur de messagerie fictif de DJANGO pour tester notre formulaire. Ceci affichera tous les e-mails envoyés par DJANGO dans le terminal. Ajoutez cette ligne au tout début du fichier `monProj/settings.py` :

```
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
```

Maintenant nous pouvons soumettre notre formulaire et regarder le résultat apparaître dans le terminal :

```
Content-Type: text/plain; charset="utf-8"
MIME-Version: 1.0
Content-Transfer-Encoding: 7bit
Subject: Message from TOF via monProj Contact form
From: cdalaigre@gmail.com
To: admin@monApp.com
Date: Wed, 13 Aug 2025 15:13:57 -0000
Message-ID: <175509803747.43335.2997547461225910119@infoens-cd2>

bonjour
```

Nous avons presque terminé, mais il y a encore une chose à faire pour améliorer la convivialité de notre formulaire... Une redirection est un type de réponse HTTP qui demande au navigateur de charger une nouvelle page. Les avantages de cette démarche sont doubles :

- Nous réduisons la probabilité d'un POST dupliqué ;
- Nous pouvons améliorer l'expérience de l'utilisateur en affichant une page de confirmation au lieu de simplement recharger la même page.

Pour implémenter la redirection, nous allons utiliser la fonction `redirect`, un raccourci pratique. Nous pouvons lui fournir un modèle d'URL avec des arguments ou directement un chemin d'URL. Ajoutons cette redirection à la fin de notre if tel que :

```
from django.shortcuts import redirect

def ContactView(request):
    titre1 = "Contact us !"

    if request.method=='POST':
        form = ContactUsForm(request.POST)
        if form.is_valid():
            send_mail(
                subject=f'Message from {form.cleaned_data["name"] or "anonyme"} via MonProjet Contact Us form',
                message=form.cleaned_data['message'],
                from_email=form.cleaned_data['email'],
                recipient_list=['admin@monprojet.com'],
            )
            return redirect('email-sent')
        else:
            form = ContactUsForm()

    return render(request, "monApp/page_home.html", {'titre1':titre1, 'form':form})
```

Veillez à ajouter l'instruction de retour à l'intérieur du bloc if `form.is_valid()`, donc en retrait de l'instruction if et non au même niveau. Lorsque le formulaire est valide et que le courriel a été envoyé, cette redirection guidera le navigateur de la page du formulaire vers une page de confirmation. C'est une page qui a un motif URL avec le nom `email-sent`. Bien entendu, cette page n'a pas encore été créée.

C'est à vous de jouer !

Vous devriez maintenant être en mesure de créer vous-même un modèle d'URL, une vue, je vous laisse donc vous en charger ! Servez vous du gabarit `page_home.html`, inutile de créer un autre template pour cette vue. Pensez à votre taux de couverture de tests...